

EDGE DETECTION & SHARPENING

Nabila Puspita Firdi, S.T., M.Sc.



Teknik Biomedik

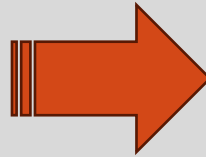
Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

OUTLINE

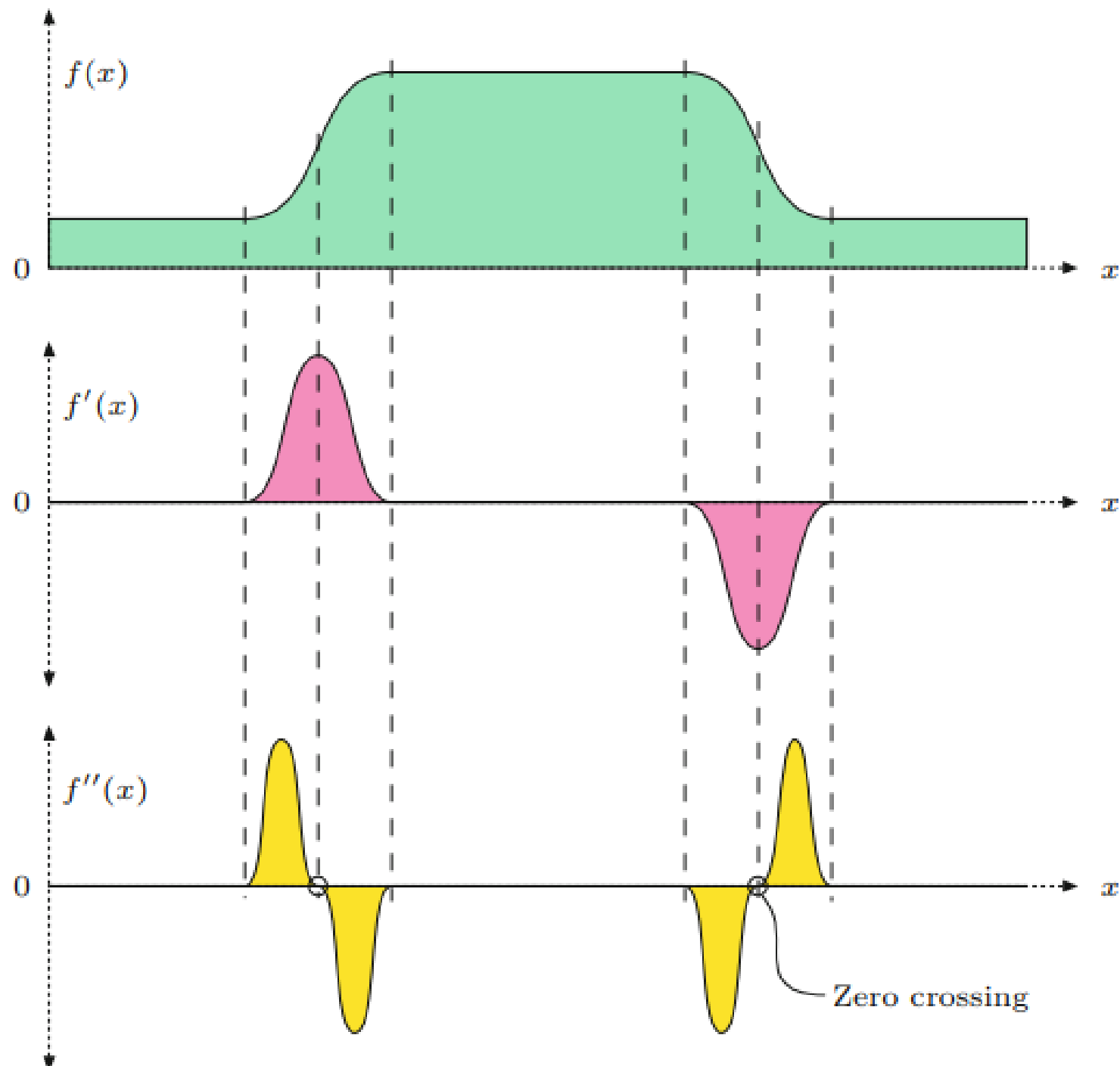
LAST WEEK (WEEK#9):

- Simple Edge Operators:
 - Prewitt Operator
 - Sobel
 - Roberts
 - Extended Sobel
 - Kirsch



THIS WEEK (WEEK#10):

- Other Edge Operator:
 - Canny Edge Operator
- Image Sharpening:
 - Laplacian
 - Unsharp masking



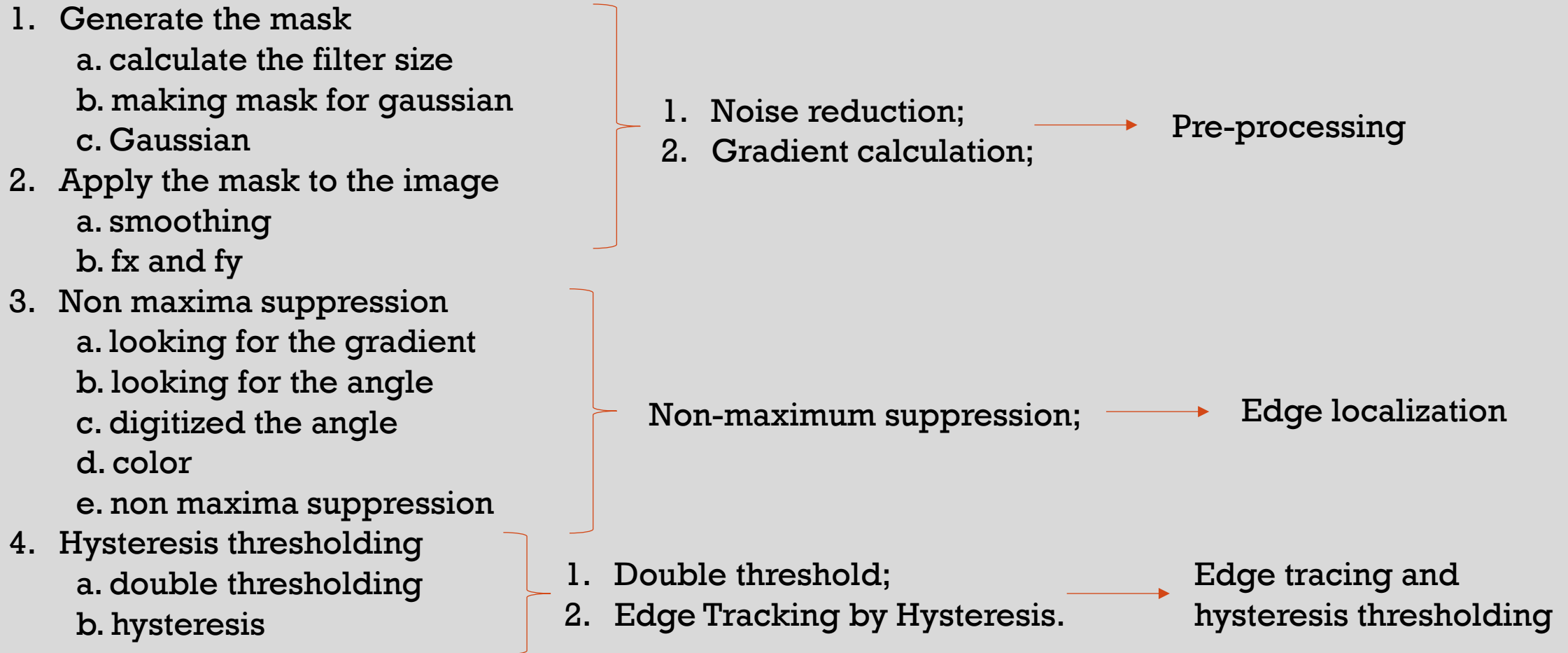
EDGE DETECTION BASED ON SECOND DERIVATIVES

- edges can be found at zero positions or-even better-at the **zero crossings** of the second derivatives.
- second derivatives → tend to **amplify image noise**
- **Solution:** presmoothing is usually applied with suitable low-pass filters
- Example: Laplacian-of-Gaussian (LoG) (re: Digital Image Processing, p.610)

CANNY EDGE OPERATORS

- **3 main goals: minimize** the number of **false edge** points, **achieve good localization** of edges, deliver only a **single mark** on each edge
- Canny “filter” → gradient method, but uses the zero crossing of second derivatives for precise edge localization
- The Canny edge detector, even in its simple (single-scale) version with an adjustable σ , remains highly effective and even outperforms classical methods.
- It produces clean, sharp, and connected edges, which greatly facilitates subsequent image analysis processes.
- Basic (single scale) form:
 1. Pre-processing
 2. Edge localization
 3. Edge tracing and hysteresis thresholding

CANNY EDGE OPERATORS

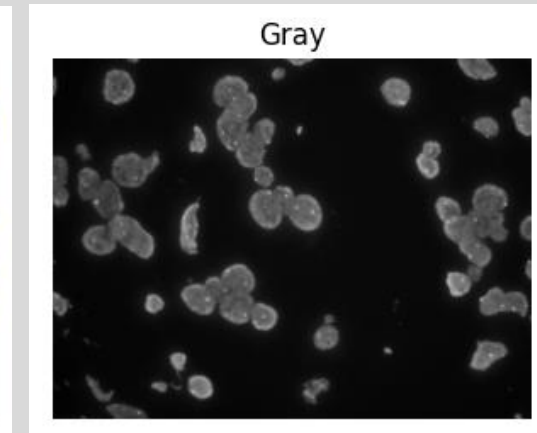
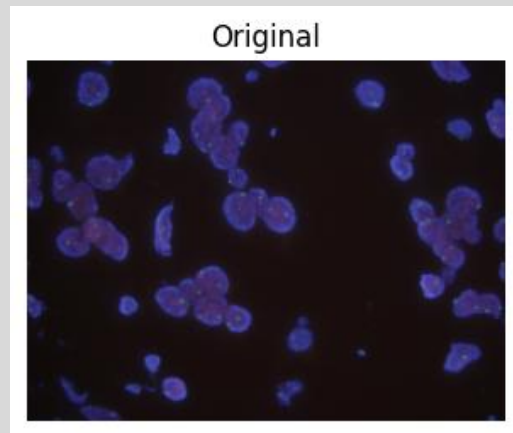


PRE-PROCESSING (1)

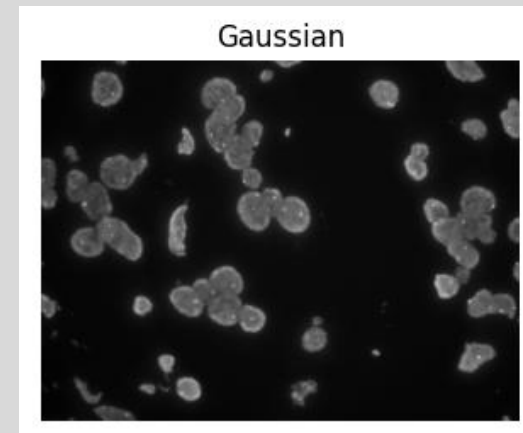
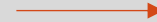
1. Generate the mask

- calculate the filter size
- making mask for gaussian
- Gaussian

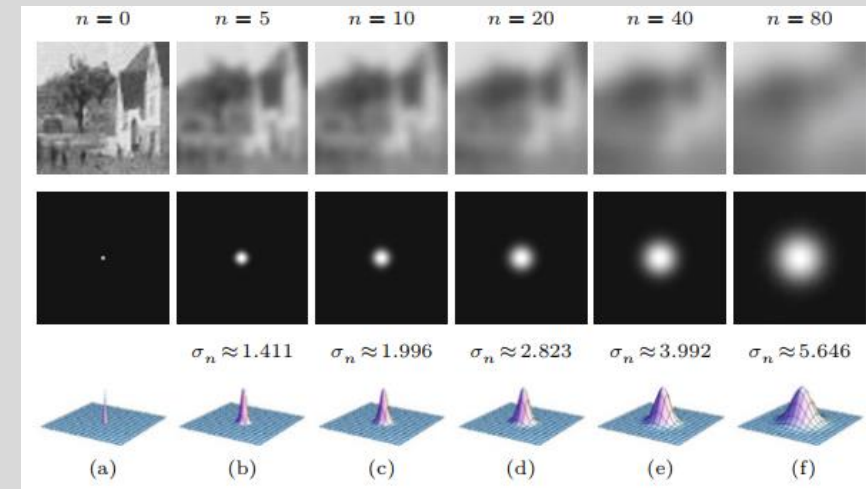
Noise reduction



I



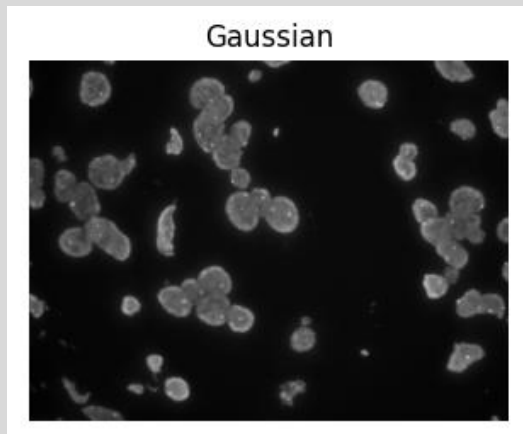
$$\bar{I} = I * H^{G, \sigma_n}$$



$$H^{G, \sigma}(x, y) = e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

PRE-PROCESSING (2)

2. Apply the mask to the image
 - a. smoothing
 - b. f_x and f_y

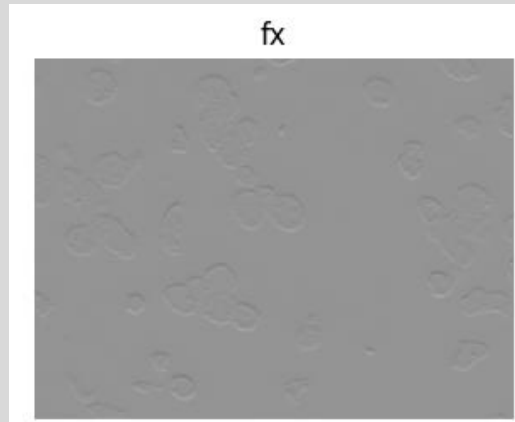


$\bar{I}$

$$H^{G,\sigma}(x,y) = e^{-\frac{x^2+y^2}{2\sigma^2}}$$

$$\bar{I}_x = \frac{\partial H^{G,\sigma}}{\partial x} = -\frac{x}{\sigma^2} \cdot \exp\left(-\frac{x^2+y^2}{\sigma^2}\right)$$

$$\bar{I}_y = \frac{\partial H^{G,\sigma}}{\partial y} = -\frac{y}{\sigma^2} \cdot \exp\left(-\frac{x^2+y^2}{\sigma^2}\right)$$



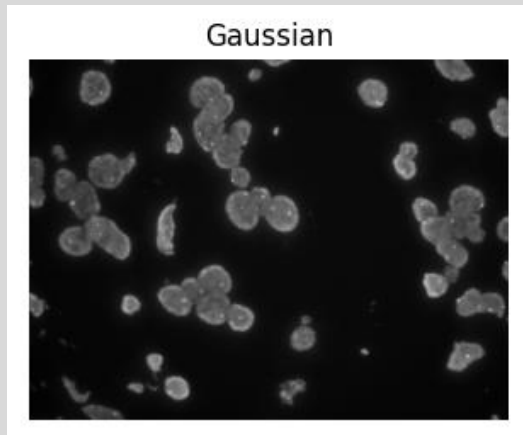
$\bar{I}_x$



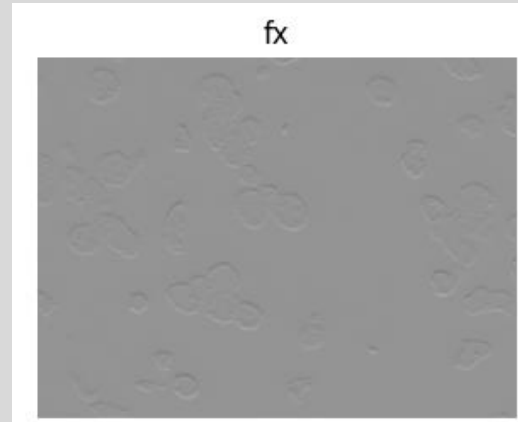
$\bar{I}_y$

EDGE LOCALIZATION(1)

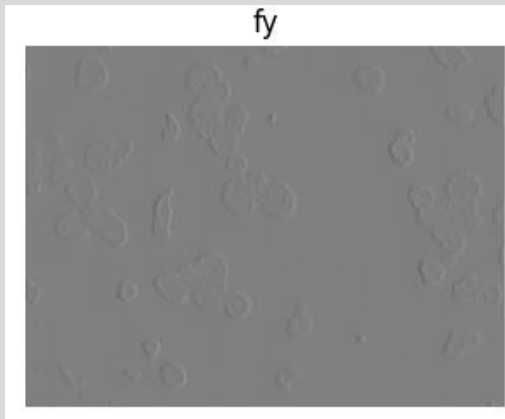
3. Non maxima suppression
 - a. looking for the gradient
 - b. looking for the angle
 - c. digitized the angle
 - d. color
 - e. non maxima suppression



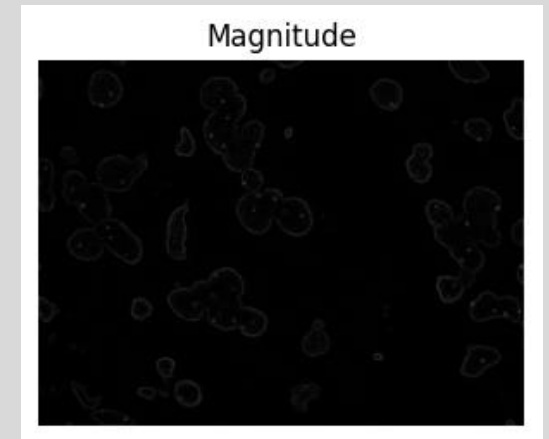
$\bar{I}$



$\bar{I}_x$



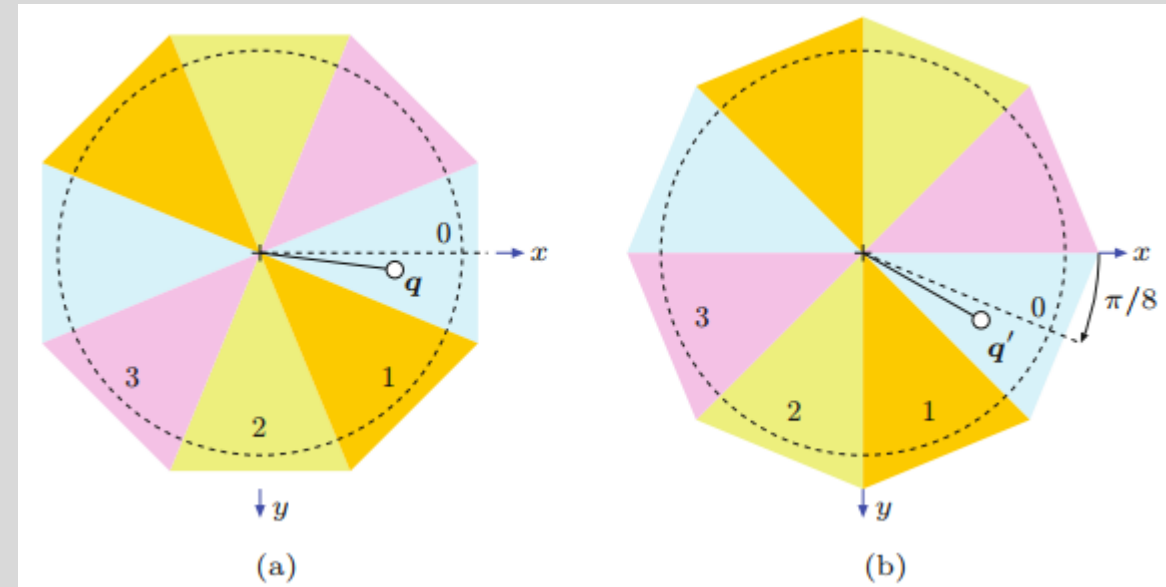
$\bar{I}_y$



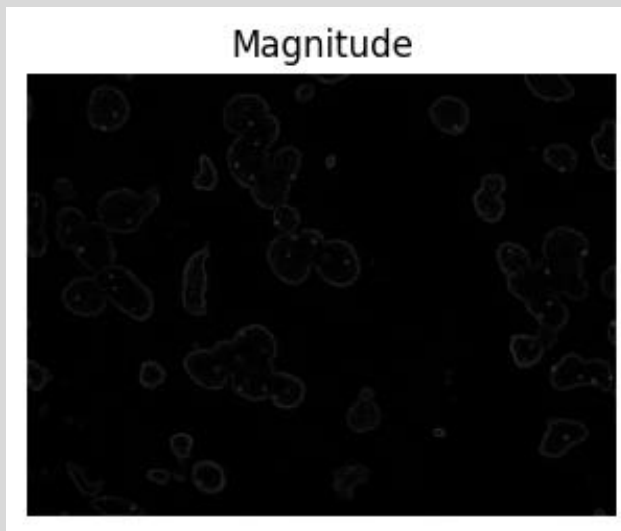
$$E_{mag} = \sqrt{\bar{I}_x^2 + \bar{I}_y^2}$$

EDGE LOCALIZATION(2)

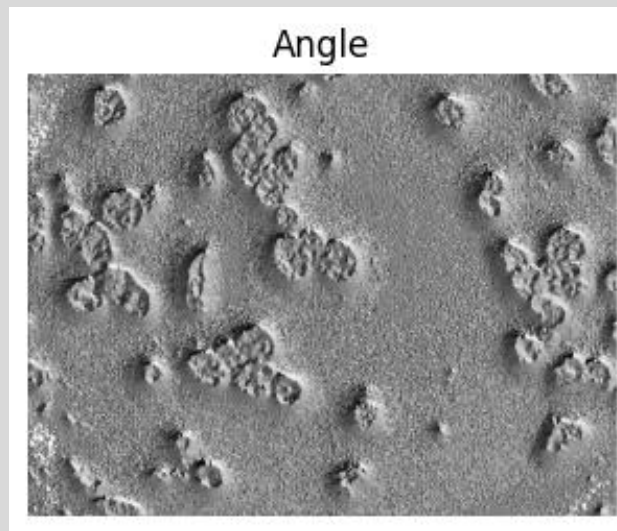
3. Non maxima suppression
 - a. looking for the gradient
 - b. looking for the angle
 - c. digitized the angle
 - d. color
 - e. non maxima suppression



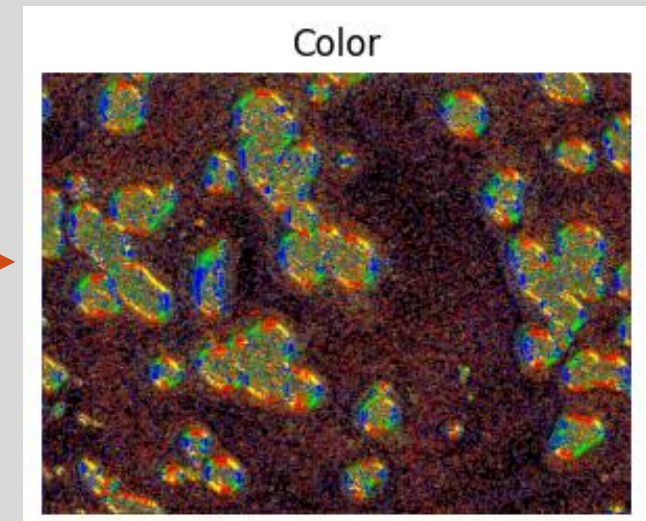
$$\Phi(u, v) = \tan^{-1} \left(\frac{\bar{I}_y(u, v)}{\bar{I}_x(u, v)} \right) = \text{ArcTan} \left(\bar{I}_x(u, v), \bar{I}_y(u, v) \right)$$



Gradient magnitude



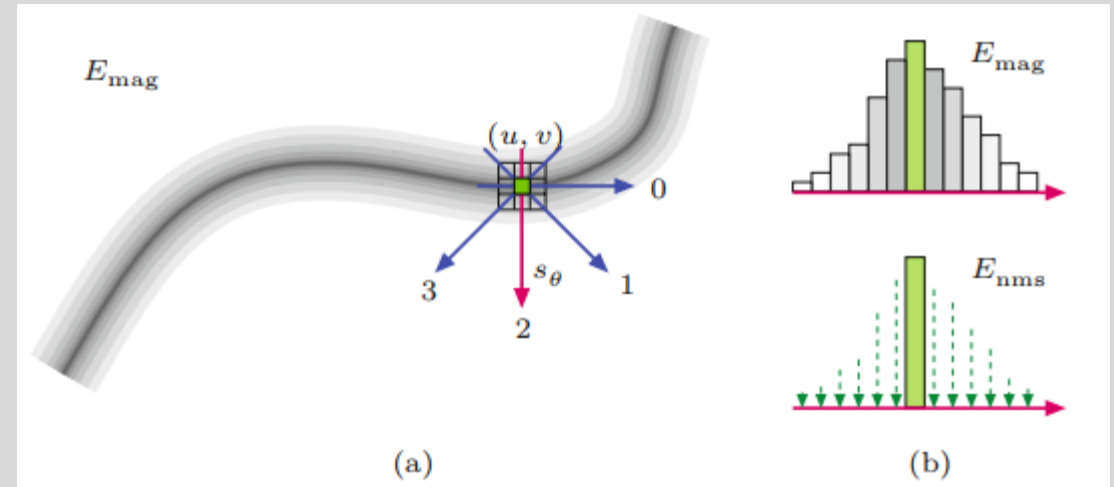
$\Phi(u, v)$



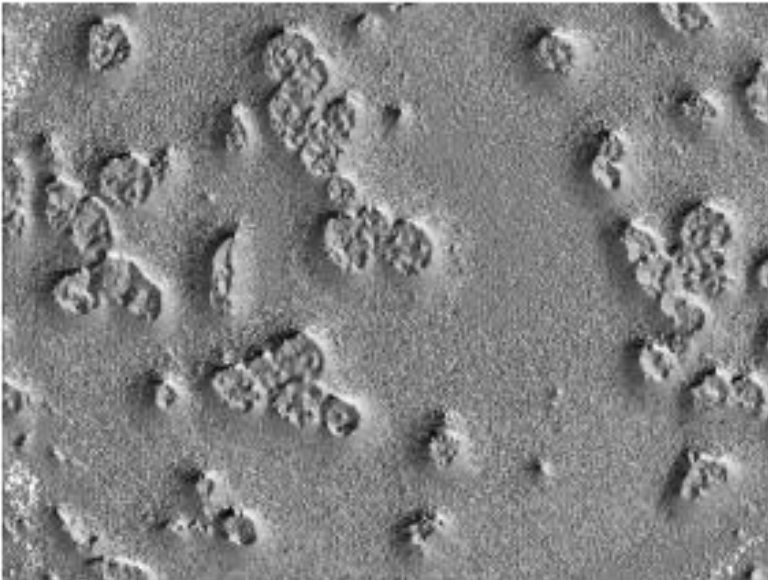
Digitized angle+color

EDGE LOCALIZATION(3)

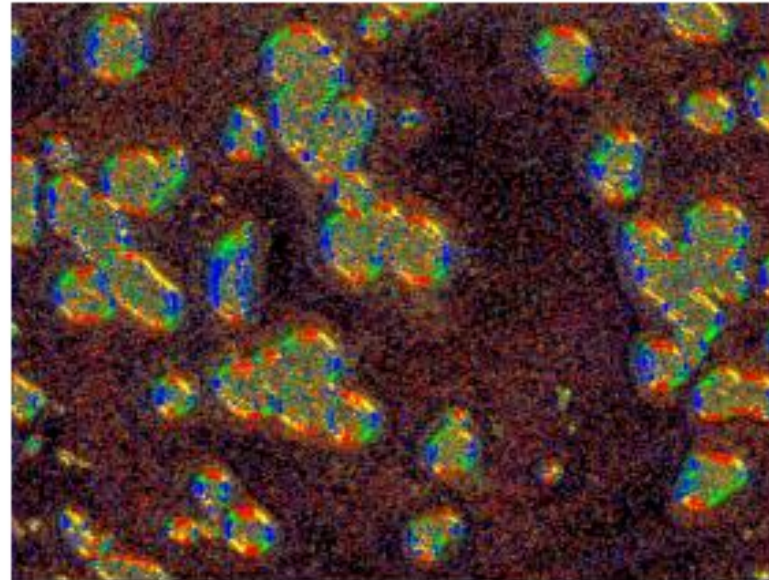
3. Non maxima suppression
 - a. looking for the gradient
 - b. looking for the angle
 - c. digitized the angle
 - d. color
 - e. non maxima suppression



Angle



Color



Non-max Supression



EDGE TRACING & HYSTERESIS THD (1)

4. Hysteresis thresholding

- double thresholding
- hysteresis

- Start with a Non-Maximum Suppressed Image
 - Use the result of the non-maximum suppression process where weak edges have already been removed, and only potential edge pixels remain (stored in the Enms map).
- Define Two Threshold Values
 - Set two thresholds:
 - thi: High threshold (e.g., 5.0 for 8-bit images or 20% of the max gradient magnitude)
 - tlo: Low threshold (e.g., 2.5 for 8-bit images or 5% of the max gradient magnitude)
 - Make sure $thi > tlo$.
- Scan the Image for Strong Edge Pixels
 - Traverse the entire Enms image to find pixels where $Enms(u, v) \geq thi$ and that have not been visited yet.
 - These are strong edge pixels and serve as starting points for edge tracing.

EDGE TRACING & HYSTERESIS THD (2)

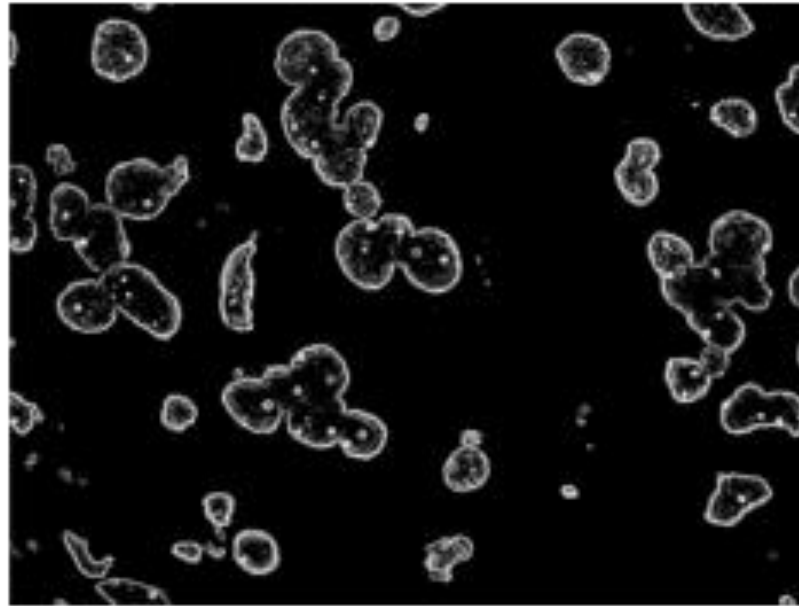
- Start a New Edge Trace
 - For every such strong edge pixel found:
 - Mark it as visited.
 - Begin edge tracing from this pixel by checking all connected neighbors (usually 8-connectivity).
- Follow Connected Pixels with Weak Edge Values
 - For each connected neighbor:
 - If its gradient magnitude $E_{nms}(u', v')$ is greater than or equal to t_{lo} , and it has not been visited yet: Add it to the current edge trace. Recursively or iteratively trace its neighbors in the same way.
- Keep Only Valid Edge Traces
 - Only preserve edge traces that contain at least one pixel $\geq t_{hi}$ and do not include any pixels $< t_{lo}$.
 - Discard traces that do not meet these conditions.
- Result: Cleaned and Connected Edge Map
 - The final output will contain connected edges that are robust to noise but preserve weak edges only when they are part of a strong structure.

EDGE TRACING & HYSTERESIS THD (3)

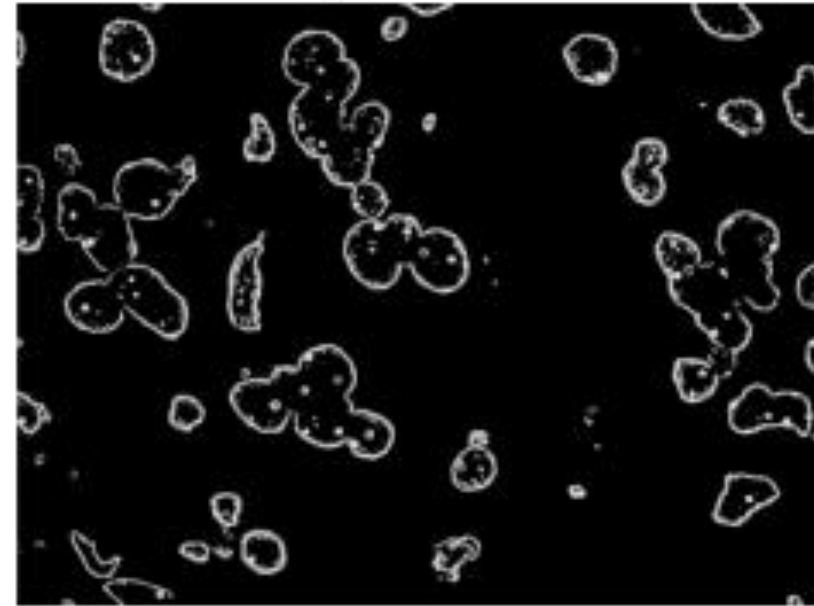
Non-max Supression



Threshold



Hysterisis



CODING TIME

```
for object to mirror  
mirror_mod.mirror_object  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True
```

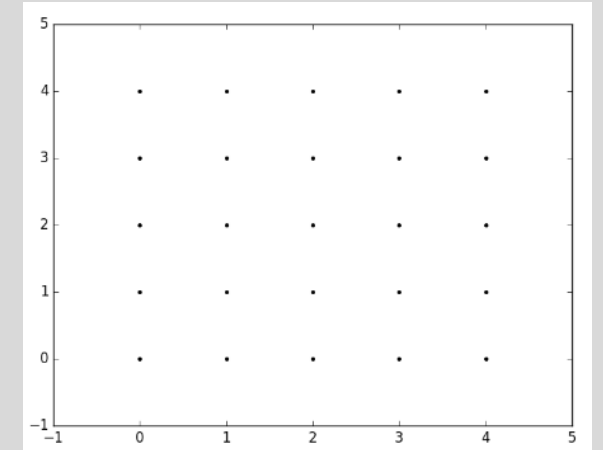
```
selection at the end -add  
ob.select= 1  
if ob.select=1  
scene.objects[ob.name]  
"selected" + str(ob.name)  
ob.select = 0  
by context.select  
data.objects[one.name].select
```

```
print("please select exactly  
-- OPERATOR CLASSES --
```

```
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"
```

CANNY EDGE DETECTION

```
sigma = 0.5  
T = 0.3  
x, y = MaskGeneration(T,sigma)  
gauss = Gaussian (x,y,sigma)
```



```
def sHalf(T, sigma):  
    temp = -np.log(T) * 2 * (sigma ** 2)  
    return np.round(np.sqrt(temp))
```

```
def calculate_filter_size(T, sigma):  
    return 2*sHalf(T, sigma) + 1
```

Define sHalf and
calculate the
kernel size

```
def MaskGeneration(T, sigma):  
    N = calculate_filter_size(T, sigma)  
    shalf = sHalf(T, sigma)  
    y, x = np.meshgrid(range(-int(shalf), int(shalf) + 1), range(-int(shalf), int(shalf) + 1))  
    return x, y
```

Define mask generation

```
def Gaussian(x,y, sigma):  
    temp = ((x ** 2) + (y ** 2)) / (2 * (sigma ** 2))  
    return (np.exp(-temp))
```

Define gaussian fuction

$$H^{G,\sigma}(x,y) = e^{-\frac{x^2+y^2}{2\sigma^2}}$$

CANNY EDGE DETECTION

$$\bar{I}_x = \frac{\partial H^{G,\sigma}}{\partial x} = -\frac{x}{\sigma^2} \cdot \exp\left(-\frac{x^2 + y^2}{\sigma^2}\right)$$

$$\bar{I}_y = \frac{\partial H^{G,\sigma}}{\partial y} = -\frac{y}{\sigma^2} \cdot \exp\left(-\frac{x^2 + y^2}{\sigma^2}\right)$$

```
def calculate_gradient_X(x,y, sigma):  
    temp = (x ** 2 + y ** 2) / (2 * sigma ** 2)  
    return -((x * np.exp(-temp)) / sigma ** 2)
```

```
def calculate_gradient_Y(x,y, sigma):  
    temp = (x ** 2 + y ** 2) / (2 * sigma ** 2)  
    return -((y * np.exp(-temp)) / sigma ** 2)
```

Calculating gaussian gradient
in x and y

```
def pad(img, kernel):  
    r, c = img.shape  
    kr, kc = kernel.shape  
    padded = np.zeros((r + kr, c + kc), dtype=img.dtype)  
    insert = np.uint((kr)//2)  
    padded[int(insert): int(insert + r), int(insert): int(insert + c)] = img  
    return padded
```

Define pad function to pad the
image

```
def smooth(img, kernel=None):  
    if kernel is None:  
        mask = np.array([[1,1,1],[1,1,1],[1,1,1]])  
    else:  
        mask = kernel  
    i, j = mask.shape  
    output = np.zeros((img.shape[0], img.shape[1]))  
    image_padded = pad(img, mask)  
    for x in range(img.shape[0]):  
        for y in range(img.shape[1]):  
            output[x, y] = (mask * image_padded[x:x+i, y:y+j]).sum() / mask.sum()  
    return output
```

```
smooth_img = smooth (img_gray, gauss)
```

Define smooth function
using average smoothing
filter (for basic)

CANNY EDGE DETECTION (CONTD.)

```
def Create_Gx(fx, fy):  
    gx = calculate_gradient_X(fx, fy, sigma)  
    gx = (gx * 255)  
    return np.around(gx)
```

Define Gx function

```
gx = -Create_Gx(x,y)  
gy = -Create_Gy(x,y)
```

```
def Create_Gy(fx, fy):  
    gy = calculate_gradient_Y(fx, fy, sigma)  
    gy = (gy * 255)  
    return np.around(gy)
```

Define Gy function

```
def ApplyMask(image, kernel):  
    i, j = kernel.shape  
    kernel = np.flipud(np.fliplr(kernel))  
    output = np.zeros_like(image)  
    image_padded = pad(image, kernel)  
    for x in range(image.shape[0]):  
        for y in range(image.shape[1]):  
            output[x, y] = (kernel * image_padded[x:x+i, y:y+j]).sum()  
    return output
```

```
fx = ApplyMask(smooth_img, gx)  
fy = ApplyMask(smooth_img, gy)  
mag = Gradient_Magnitude(fx,fy)  
mag = mag.astype(int)
```

Applying the mask function and define gradient magnitude function

```
def Gradient_Magnitude(fx, fy):  
    mag = np.zeros((fx.shape[0], fx.shape[1]))  
    mag = np.sqrt((fx ** 2) + (fy ** 2))  
    mag = mag * 100 / mag.max()  
    return np.around(mag)
```

$$E_{mag} = \sqrt{\bar{I}_x^2 + \bar{I}_y^2}$$

CANNY EDGE DETECTION (CONTD.)

```
def Gradient_Direction(fx, fy):  
    g_dir = np.zeros((fx.shape[0], fx.shape[1]))  
    g_dir = np.rad2deg(np.arctan2(fy, fx)) + 180  
    return g_dir
```

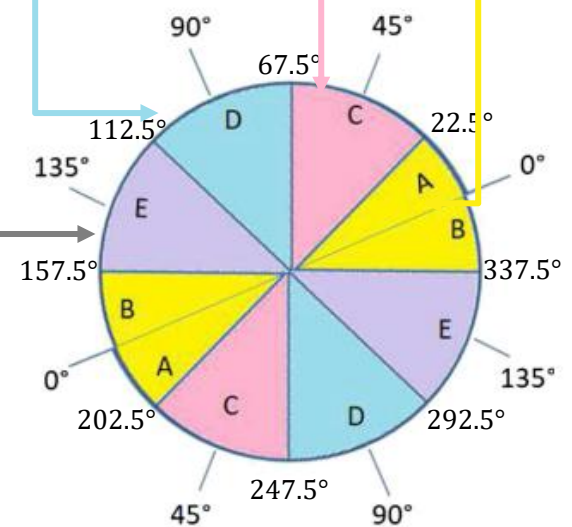
Define gradient direction

$$\Phi(u, v) = \tan^{-1} \left(\frac{\bar{I}_y(u, v)}{\bar{I}_x(u, v)} \right) = \text{ArcTan}(\bar{I}_x(u, v), \bar{I}_y(u, v))$$

```
def Digitize_angle(Angle):  
    quantized = np.zeros((Angle.shape[0], Angle.shape[1]))  
    for i in range(Angle.shape[0]):  
        for j in range(Angle.shape[1]):  
            if 0 <= Angle[i, j] <= 22.5 or 157.5 <= Angle[i, j] <= 202.5 or 337.5 < Angle[i, j] < 360:  
                quantized[i, j] = 0  
            elif 22.5 <= Angle[i, j] <= 67.5 or 202.5 <= Angle[i, j] <= 247.5:  
                quantized[i, j] = 1  
            elif 67.5 <= Angle[i, j] <= 112.5 or 247.5 <= Angle[i, j] <= 292.5:  
                quantized[i, j] = 2  
            elif 112.5 <= Angle[i, j] <= 157.5 or 292.5 <= Angle[i, j] <= 337.5:  
                quantized[i, j] = 3  
    return quantized
```

Digitized the angle

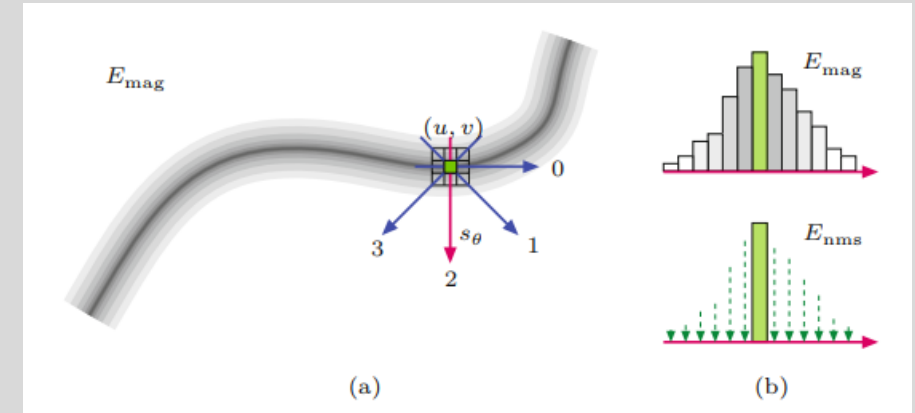
```
Angle = Gradient_Direction(fx,fy)  
quantized = Digitize_angle(Angle)
```



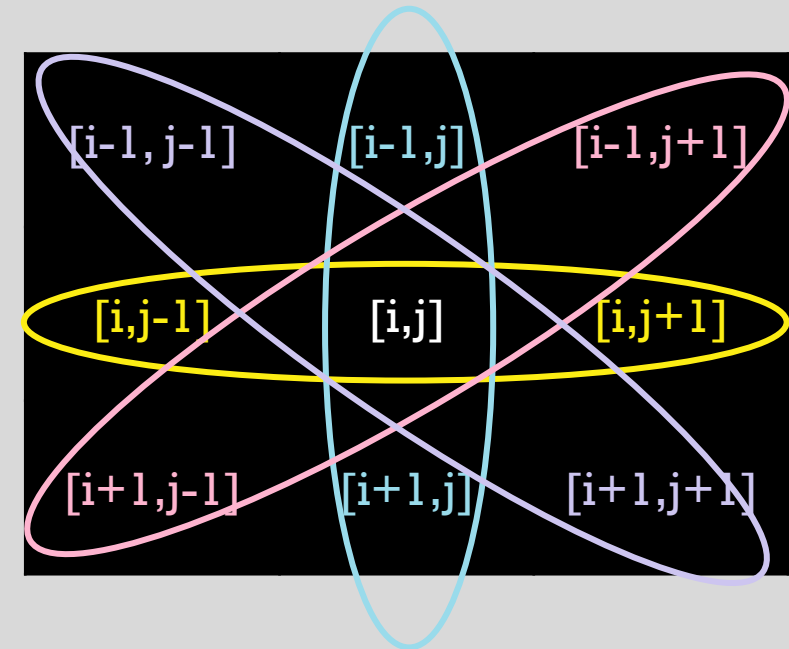
CANNY EDGE DETECTION (CONTD.)

`nms = Non_Max_Supp(quantized, Angle, mag)`

```
def Non_Max_Supp(qn, magni, D):
    M = np.zeros(qn.shape)
    a, b = np.shape(qn)
    for i in range(a-1):
        for j in range(b-1):
            if qn[i,j] == 0:
                if magni[i,j-1] < magni[i,j] or magni[i,j] > magni[i,j+1]:
                    M[i,j] = D[i,j]
                else:
                    M[i,j] = 0
            if qn[i,j] == 1:
                if magni[i-1,j+1] <= magni[i,j] or magni[i,j] >= magni[i+1,j-1]:
                    M[i,j] = D[i,j]
                else:
                    M[i,j] = 0
            if qn[i,j] == 2:
                if magni[i-1,j] <= magni[i,j] or magni[i,j] >= magni[i+1,j]:
                    M[i,j] = D[i,j]
                else:
                    M[i,j] = 0
            if qn[i,j] == 3:
                if magni[i-1,j-1] <= magni[i,j] or magni[i,j] >= magni[i+1,j+1]:
                    M[i,j] = D[i,j]
                else:
                    M[i,j] = 0
    return M
```



Define non max suppression



CANNY EDGE DETECTION (CONTD.)

```
def color(quant, mag):  
    color = np.zeros((mag.shape[0], mag.shape[1], 3), np.uint8)  
    a, b = np.shape(mag)  
    for i in range(a-1):  
        for j in range(b-1):  
            if quant[i,j] == 0:  
                if mag[i,j] != 0:  
                    color[i,j,0] = 255  
                else:  
                    color[i,j,0] = 0  
            if quant[i,j] == 1:  
                if mag[i,j] != 0:  
                    color[i,j,1] = 255  
                else:  
                    color[i,j,1] = 0  
            if quant[i,j] == 2:  
                if mag[i,j] != 0:  
                    color[i,j,2] = 255  
                else:  
                    color[i,j,2] = 0  
            if quant[i,j] == 3:  
                if mag[i,j] != 0:  
                    color[i,j,0] = 255  
                    color[i,j,1] = 255  
                else:  
                    color[i,j,0] = 0  
                    color[i,j,1] = 0  
    return color
```

```
colorized = color(quantized, mag)
```

Define the color for each direction

CANNY EDGE DETECTION (CONTD.)

```
def _double_thresholding(g_suppressed, low_threshold, high_threshold):  
    g_thresholderd = np.zeros(g_suppressed.shape)  
    for i in range(0, g_suppressed.shape[0]): # loop over pixels  
        for j in range(0, g_suppressed.shape[1]):  
            if g_suppressed[i,j] < low_threshold: # lower than low threshold  
                g_thresholderd[i,j] = 0  
            elif g_suppressed[i,j] >= low_threshold and g_suppressed[i,j] < high_threshold:  
                g_thresholderd[i,j] = 128  
            else: # higher than high threshold  
                g_thresholderd[i,j] = 255  
    return g_thresholderd  
  
def _hysteresis(g_thresholderd):  
    g_strong = np.zeros(g_thresholderd.shape)  
    for i in range(0, g_thresholderd.shape[0]): # loop over pixels  
        for j in range(0, g_thresholderd.shape[1]):  
            val = g_thresholderd[i,j]  
            if val == 128: # check if weak edge connected to strong  
                if g_thresholderd[i-1,j] == 255 or g_thresholderd[i+1,j] == 255 or g_thresholderd[i-1,j-1] == 255:  
                    g_strong[i,j] = 255 # replace weak edge as strong  
            elif val == 255:  
                g_strong[i,j] = 255 # strong edge remains as strong edge  
    return g_strong
```

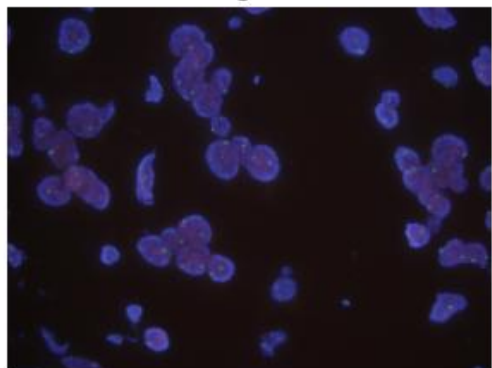
```
threshold = _double_thresholding(nms, 30, 60)  
hys = _hysteresis(threshold)
```

Define the double thresholding

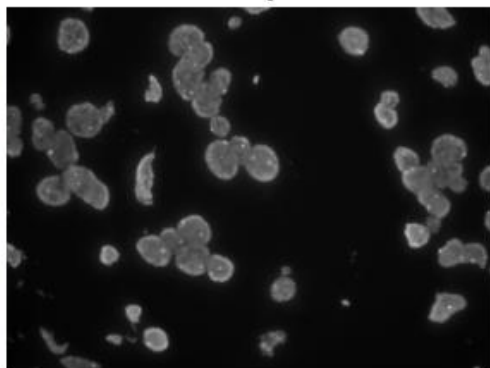
Define hysteresis / edge tracking

CANNY EDGE DETECTION (CONTD.)

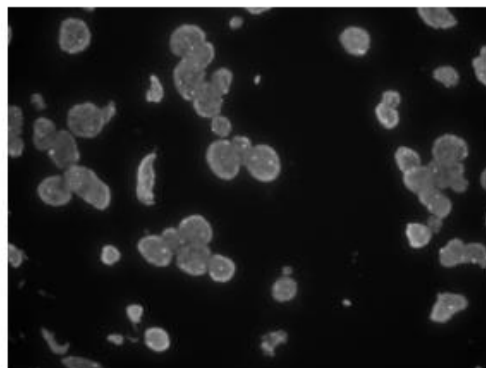
Original



Gray



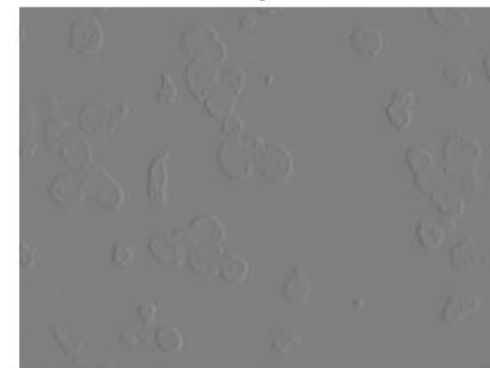
Gaussian



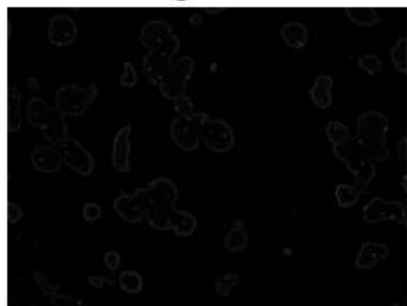
f_x



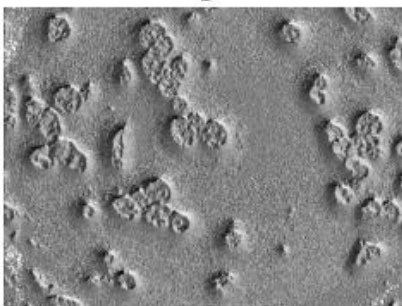
f_y



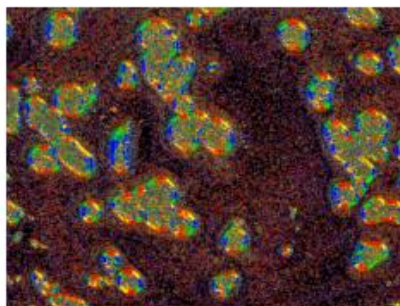
Magnitude



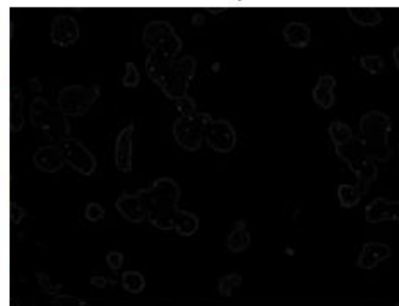
Angle



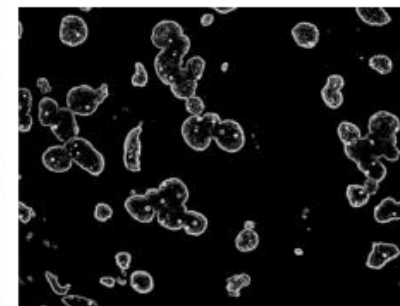
Color



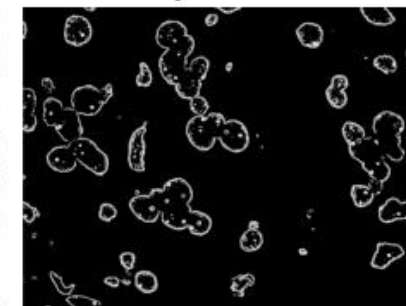
Non-max Supression



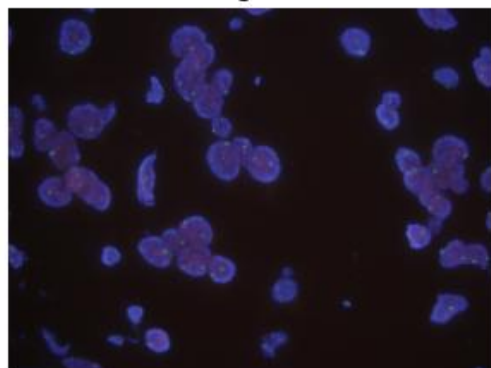
Threshold



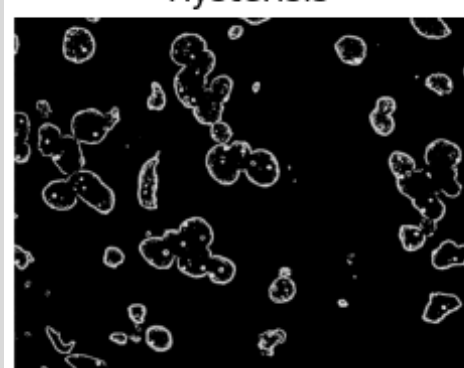
Hysteresis



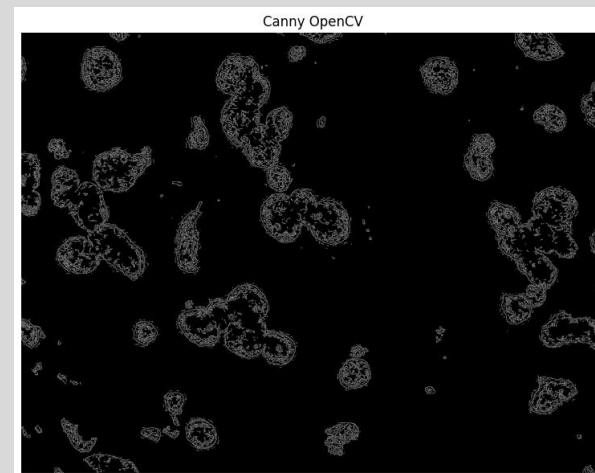
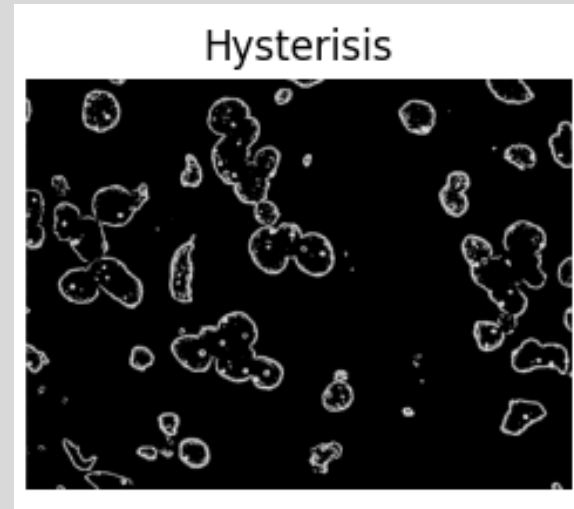
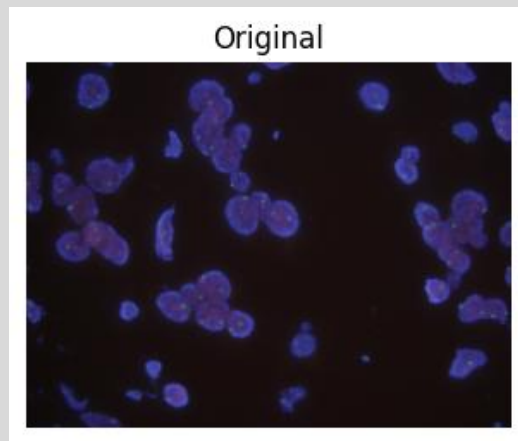
Original



Hysteresis

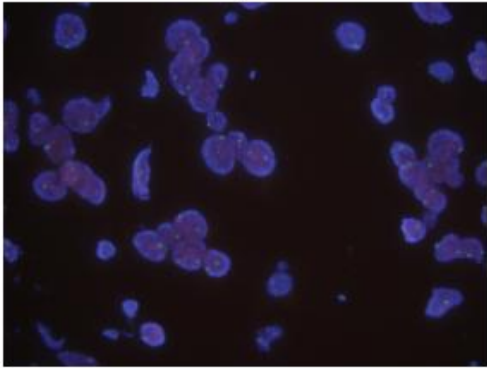


EDGE OPERATORS COMPARISON

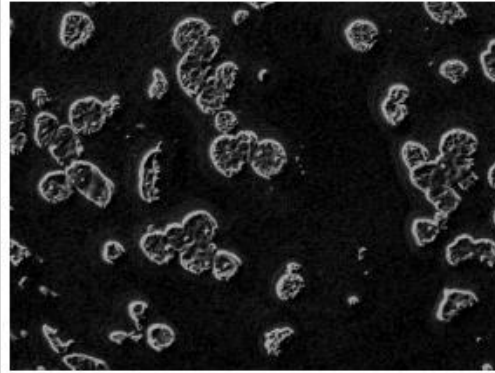


EDGE OPERATORS COMPARISON

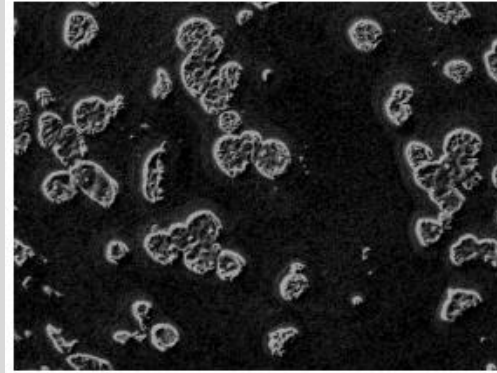
Original



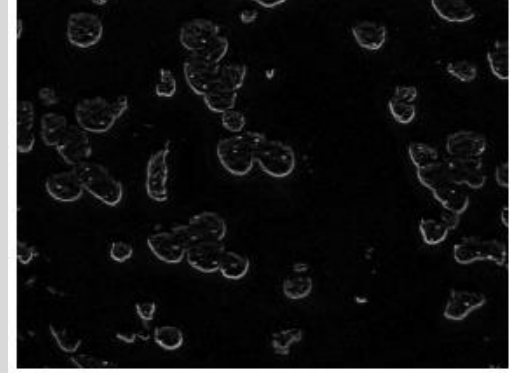
Prewitt



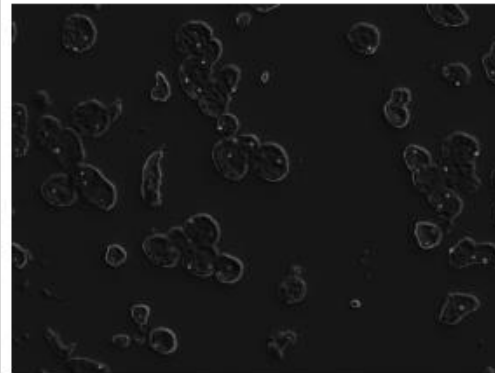
Sobel



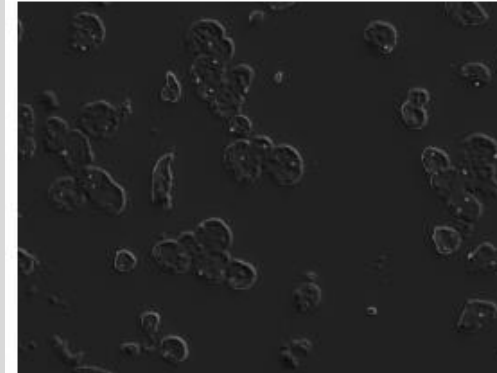
Roberts



Extended Sobel



Kirsch



Hysteresis

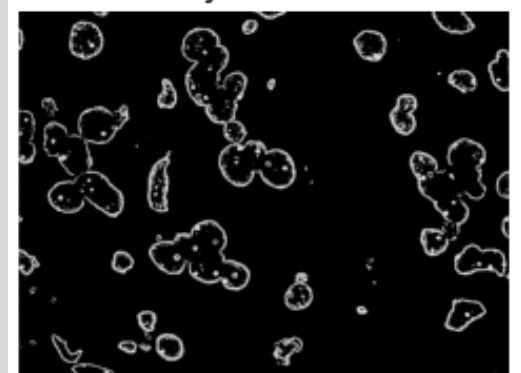
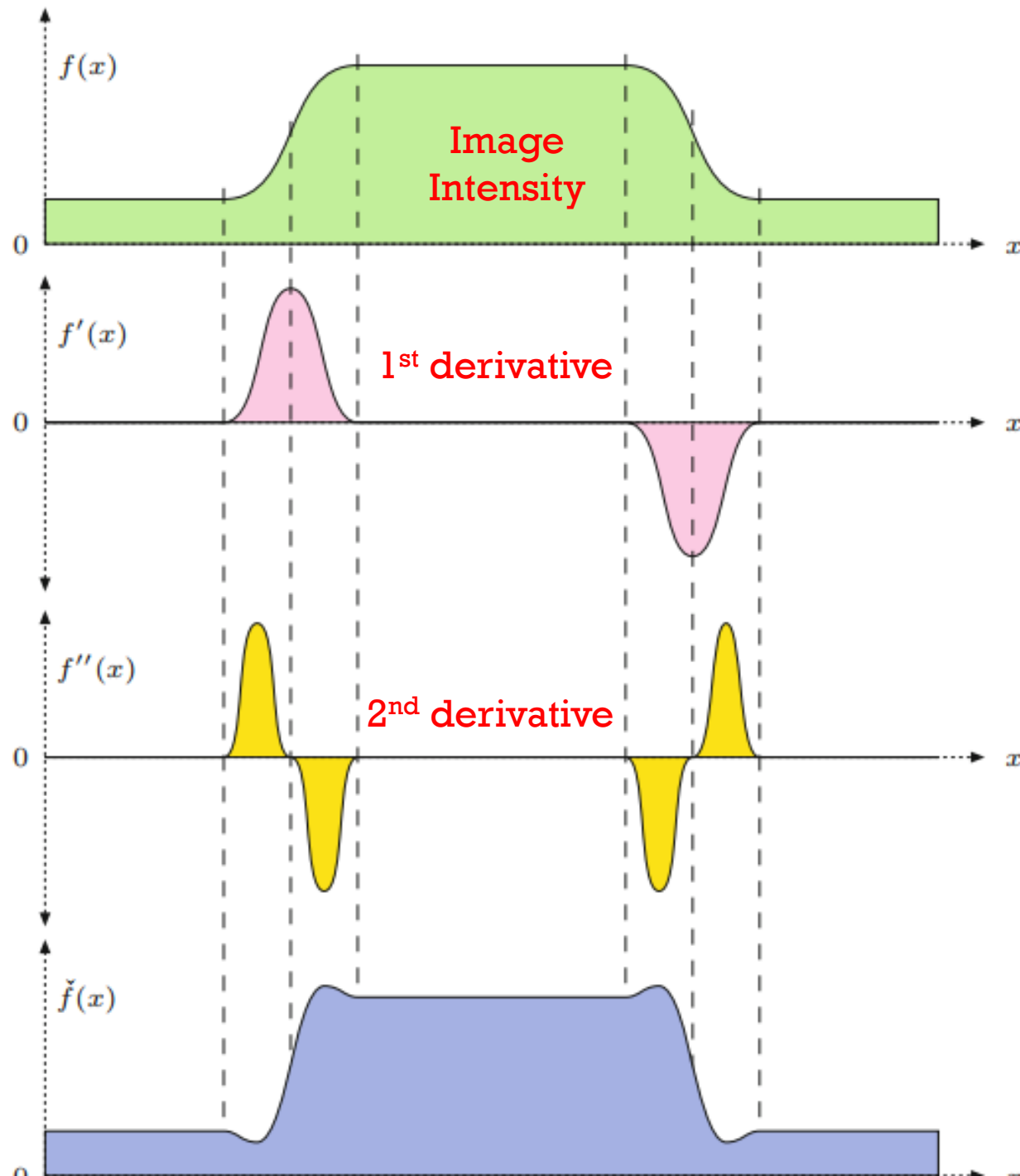


IMAGE SHARPENING

IMAGE SHARPENING

- Blurring may occur during image scanning or scaling
- Sharpening reduces effect of blurring
- How? Amplify high frequency components
- High frequencies occur at edges → rapid brightness changes
- Two main approaches:
 - Laplace filter
 - Unsharp masking

EDGE SHARPENING USING LAPLACE FILTER



$$\tilde{f}(x) = f(x) - w \cdot f''(x)$$

The diagram illustrates the components of the Laplace filter equation:

- $f(x)$ is labeled **Image Intensity** (green box).
- w is labeled **weight** (blue box).
- $f''(x)$ is labeled **2nd derivative** (yellow box).

LAPLACIAN OPERATOR

- Sharpening of 2D function $\rightarrow$ second derivatives in the horizontal and vertical direction combined $\rightarrow$ **Laplacian operator**
- **Laplacian operator** ∇^2 : the sum of the second partial derivatives along the x and y directions

$$(\nabla^2 f)(x, y) = \frac{\partial^2 f}{\partial^2 x}(x, y) + \frac{\partial^2 f}{\partial^2 y}(x, y).$$

$\downarrow$
2nd derivative of intensity in x direction

$\downarrow$
2nd derivative of intensity in y direction

$$\frac{\partial^2 f}{\partial^2 x} \approx H_x^L = \begin{bmatrix} 1 & -2 & 1 \end{bmatrix} \quad \text{and} \quad \frac{\partial^2 f}{\partial^2 y} \approx H_y^L = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} \quad \rightarrow \quad H^L = H_x^L + H_y^L = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

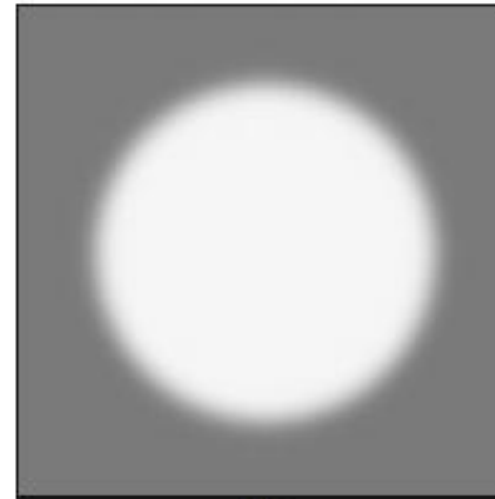
LAPLACIAN OPERATOR

$$H_8^L = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad \text{oder} \quad H_{12}^L = \begin{bmatrix} 1 & 2 & 1 \\ 2 & -12 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

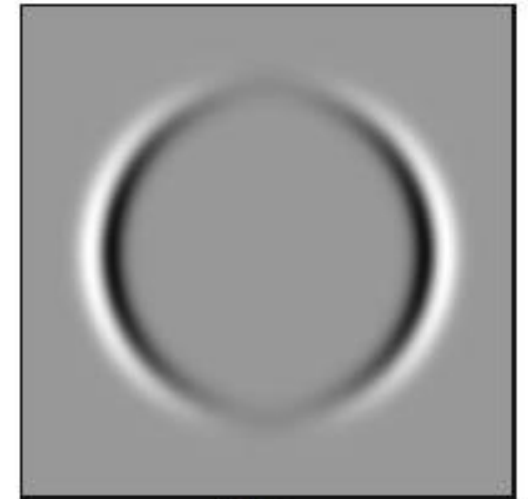
$$I * H^L = I * (H_x^L + H_y^L) = (I * H_x^L) + (I * H_y^L) = I_{xx} + I_{yy}.$$

Sharpening:

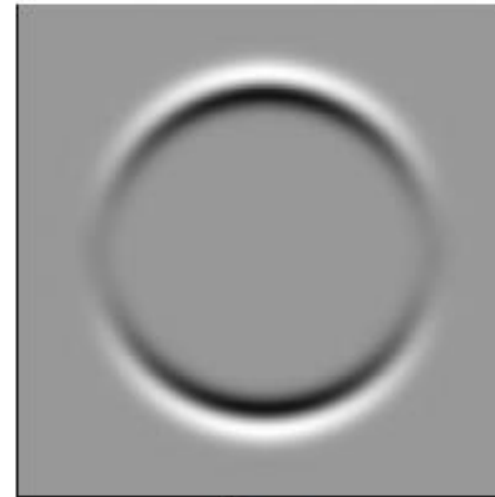
$$I' \leftarrow I - w \cdot (H^L * I).$$



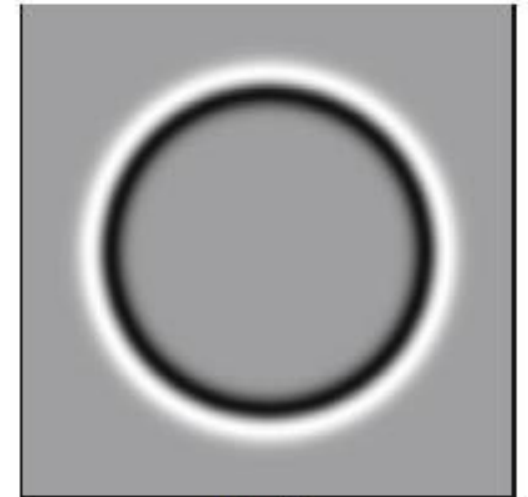
(a) I



(b) I_{xx}



(c) I_{yy}



(d) $\nabla^2 I$

CODING TIME

```
for object to mirror_mod.mirror_object
operation == "MIRROR_X":
mirror_mod.use_x = True
mirror_mod.use_y = False
mirror_mod.use_z = False
operation == "MIRROR_Y":
mirror_mod.use_x = False
mirror_mod.use_y = True
mirror_mod.use_z = False
operation == "MIRROR_Z":
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True
```

```
@selection at the end -add
mirror_ob.select= 1
mirror_ob.select=1
mirror_ob.scene.objects.active = mirror_ob
["selected" + str(mirror_ob.name)]
mirror_ob.select = 0
py.context.selected_objects[0] = mirror_ob
data.objects[one.name].select = 1
```

```
print("please select exactly one object")
```

```
-- OPERATOR CLASSES --
```

```
types.Operator):
X mirror to the selected
object.mirror_mirror_x"
mirror X"
```

HOW TO CODE LAPLACIAN OPERATOR?

- Import library
- Import image
- RGB $\rightarrow$ Gray
- Smoothing image (Gaussian blur)
- Convolute Image with the kernel
- Subtracting the original image with the convoluted image multiplied by weight

LAPLACIAN OPERATOR

```
HL = np.array([[0.0, 1.0, 0.0], [1.0, -4.0, 1.0], [0.0, 1.0, 0.0]])
HL_x = np.array([1.0, -2.0, 1.0])
HL_y = np.array([[1.0], [-2.0], [1.0]])
w = 1.0
```

Define the Laplacian kernel and weight

```
def laplacian_x (image):
    x = cv2.filter2D(image, -1, HL_x)
    x_abs = np.abs(x)
    return cv2.normalize(x_abs, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8UC1)

def laplacian_y (image):
    y = cv2.filter2D(image, -1, HL_y)
    y_abs = np.abs(y)
    return cv2.normalize(y_abs, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8UC1)

def laplacianXY (image):
    xy = cv2.filter2D(image, -1, HL)
    xy_abs = np.abs(xy)
    return cv2.normalize(xy_abs, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8UC1)

def to_uint8(img):
    return np.clip(img, 0, 255).astype(np.uint8)
```

Define the convolution function and normalization

LAPLACIAN OPERATOR

```
# Convert image to grayscale if it's not already
if len(image.shape) == 3:
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
else:
    gray_image = image

blur = cv2.GaussianBlur(gray_image, (3, 3), 0)

##Laplacian
x_laplacian = laplacian_x(blur)
y_laplacian = laplacian_y(blur)
xy_laplacian = x_laplacian + y_laplacian
xy_sharpening = gray_image - w*xy_laplacian

laplacian_hasil = laplacianXY(blur)
sharpening_hasil = gray_image - w*laplacian_hasil
```

Converting image to
grayscale and
smoothen the image

Laplacian operator
process

LAPLACIAN OPERATOR (CONTD.)

```
HL = np.array([[0.0, 1.0, 0.0], [1.0, -4.0, 1.0], [0.0, 1.0, 0.0]])  
HL_x = np.array([1.0, -2.0, 1.0])  
HL_y = np.array([1.0, -2.0, 1.0])
```

Define the Laplacian kernel based on:

$$H^L = H_x^L + H_y^L = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\frac{\partial^2 f}{\partial^2 x} \approx H_x^L = \begin{bmatrix} 1 & -2 & 1 \end{bmatrix} \quad \text{and} \quad \frac{\partial^2 f}{\partial^2 y} \approx H_y^L = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

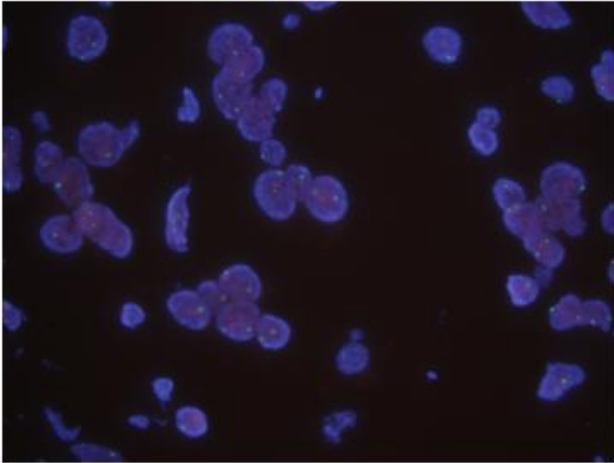
```
laplacian_hasil = laplacianXY(blur)  
sharpening_hasil = gray_image - w*laplacian_hasil
```

Calculate the sharpening image by subtracting the original image with weight and the convolved image using equation:

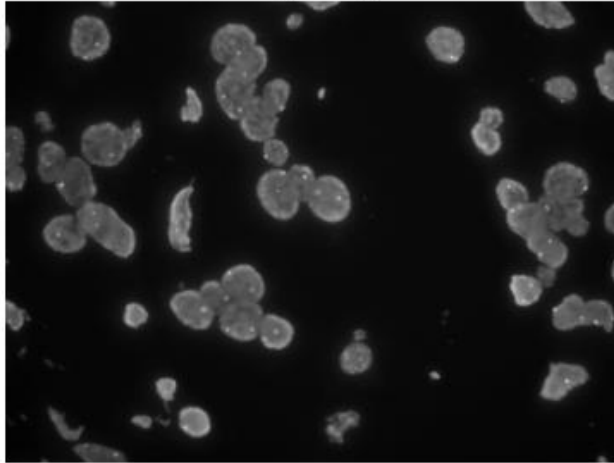
$$I' \leftarrow I - w \cdot (H^L * I).$$

LAPLACIAN OPERATOR RESULTS

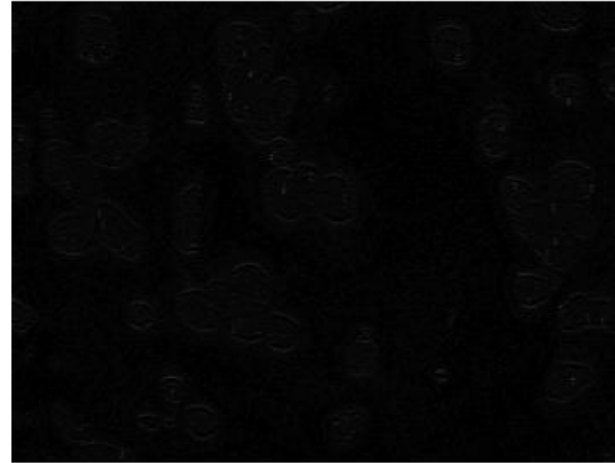
Original



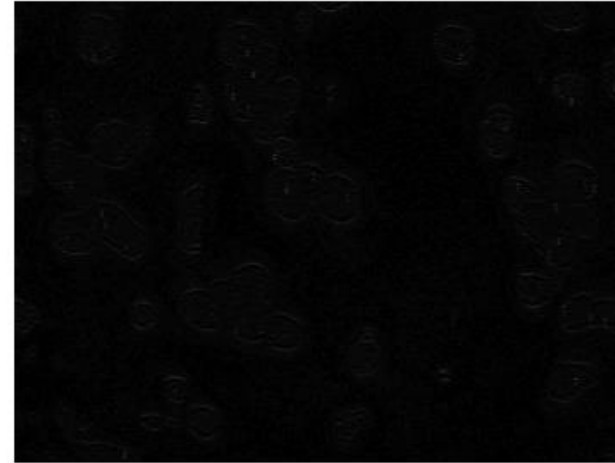
Gray Image



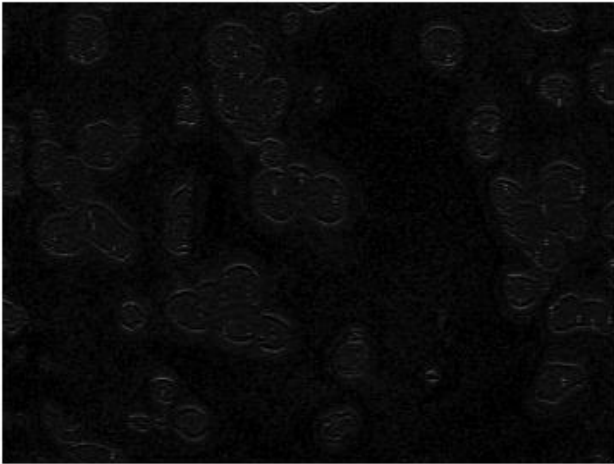
Laplacian X



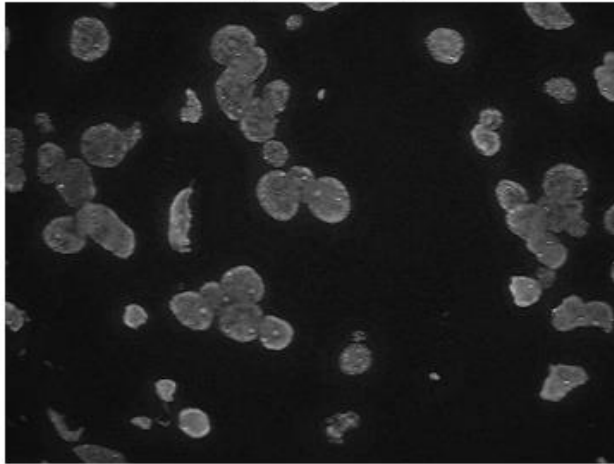
Laplacian Y



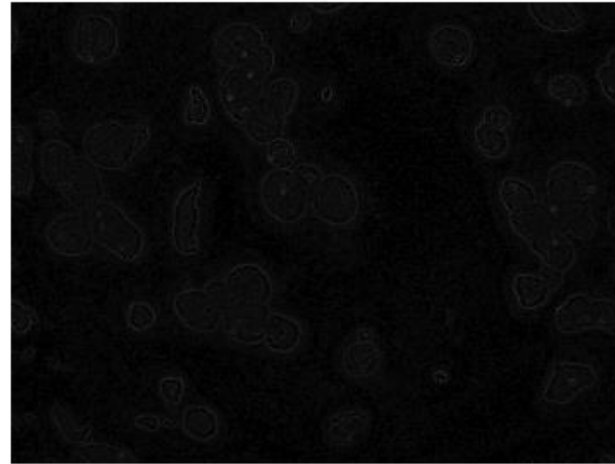
Laplacian XY



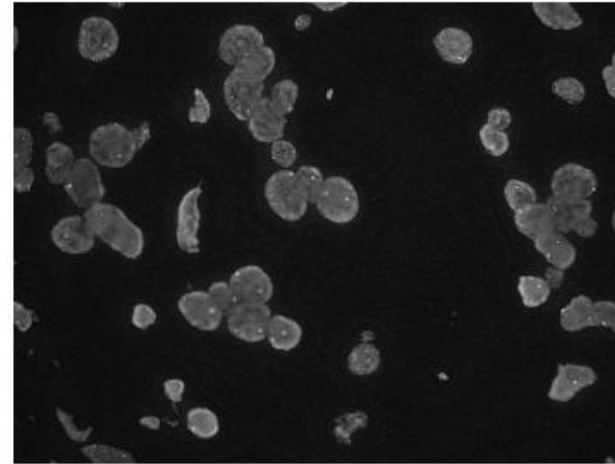
XY_sharpening



Hasil laplacian

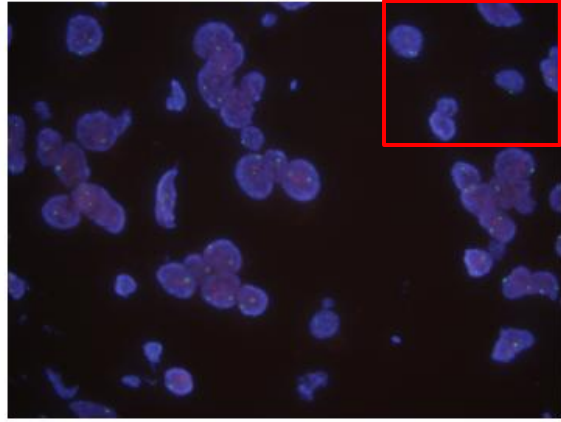


Sharpening Hasil

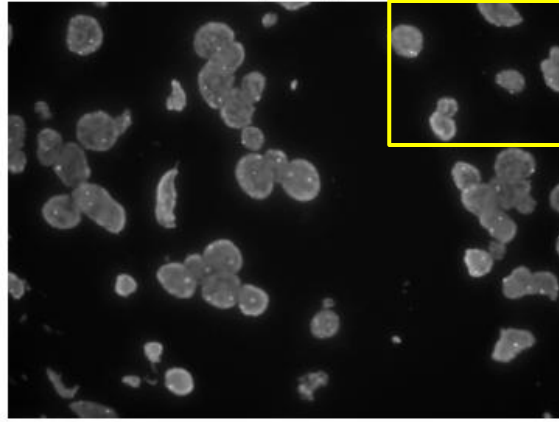


LAPLACIAN EDGE SHARPENING RESULTS

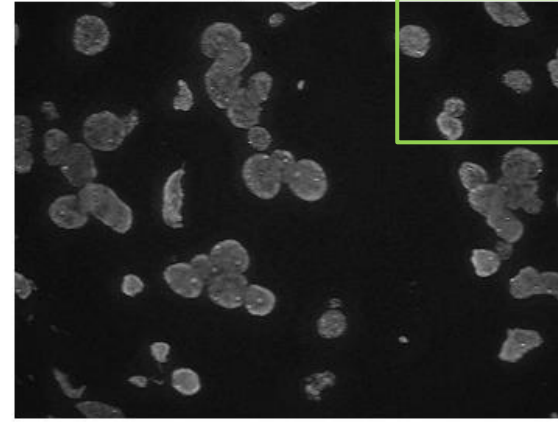
Original



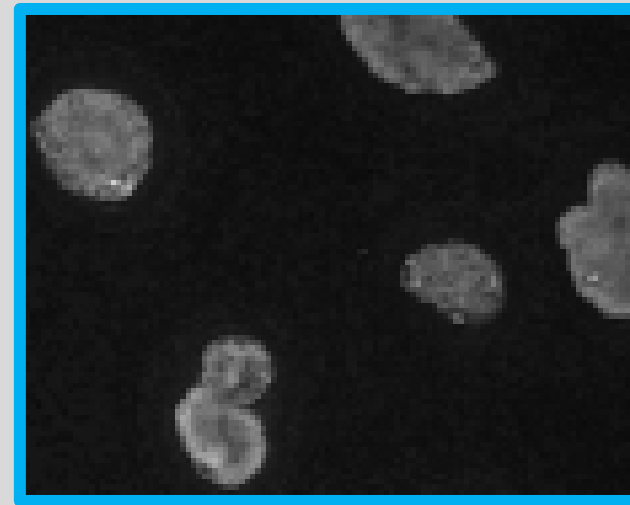
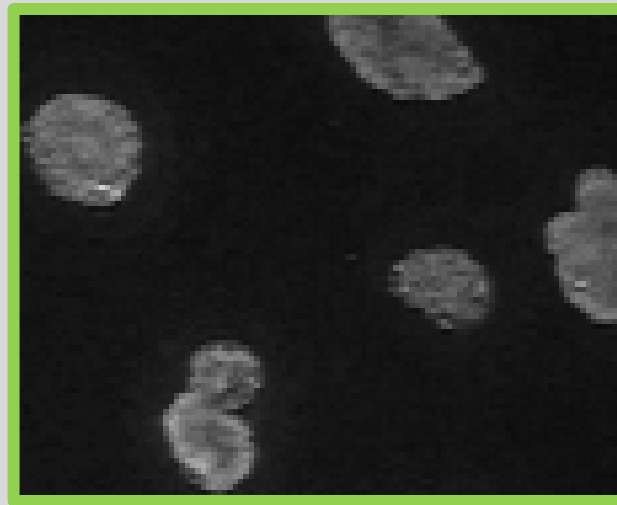
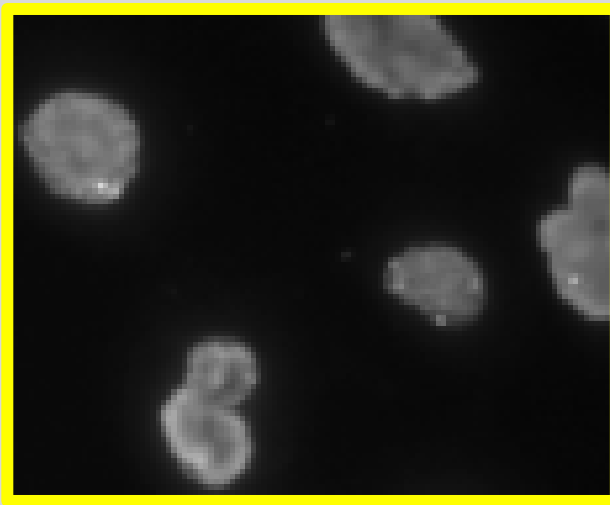
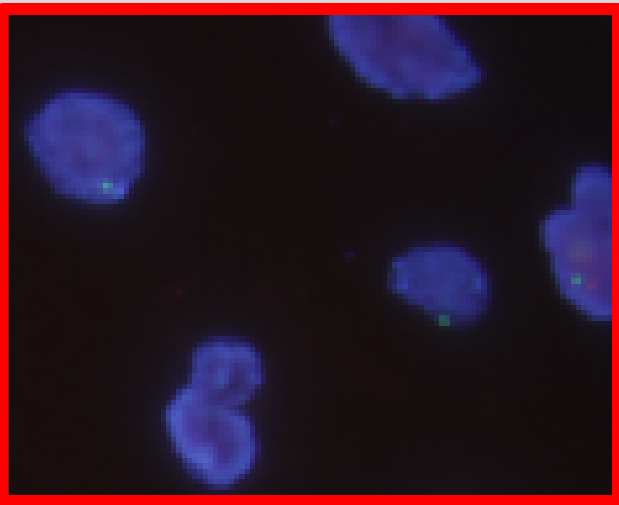
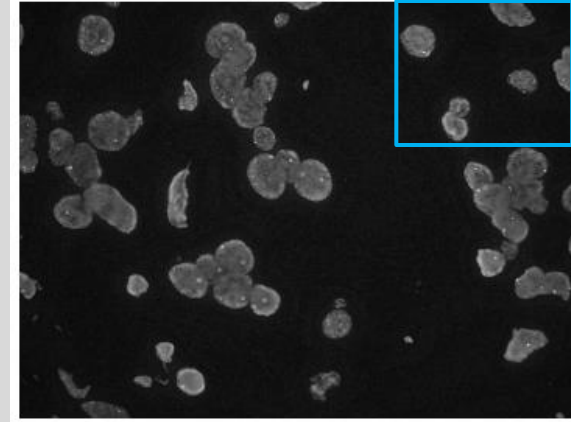
Gray Image



XY_sharpening



Sharpening Hasil



UNSHARP MASKING

- Sharpening that is particularly popular in astronomy, digital printing, etc.
- The term originates from classical photography, where the sharpness of an image was optically enhanced by combining it with a smoothed (“unsharp”) copy.
- Unsharp masking (USM) process:
 - Create mask image M by subtracting a smoothed version of I , obtained by filtering $\tilde{H}$

$$M \leftarrow I - (I * \tilde{H}) = I - \tilde{I}$$

- The kernel $\tilde{H}$ of the smoothing filter is assumed to be normalized
- To obtain the sharpened image $\check{I}$, the mask M is added to the original image I , weighted by a factor a , which controls the amount of sharpening

$$\check{I} \leftarrow I + a \cdot M$$

- And thus,

$$\check{I} \leftarrow I + a \cdot (I - \tilde{I}) = (1 + a) \cdot I - a \cdot \tilde{I}$$

UNSHARP MASKING (CONTD.)

```
def unsharp_mask(img, a, kernel_size=(5,5), sigma=1.0):  
    (w,h) = img.shape  
    unsharp = np.zeros((w,h))  
    blurred = cv2.GaussianBlur(img, kernel_size, sigma)  
    mask = img - blurred  
    unsharp = mask + (img*a)  
    return unsharp
```

```
hasil_unsharp = unsharp_mask(img_gray, 0.7, (5,5), 1.0)  
fig1, axes = plt.subplots(1,2, figsize=(20,5))
```

Define the mask function by following equation:

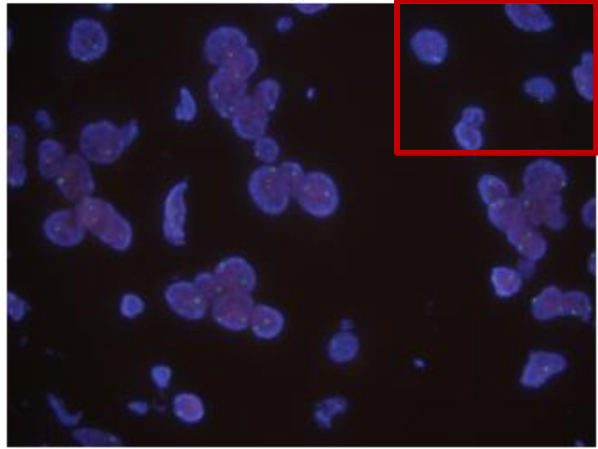
$$M \leftarrow I - (I * \tilde{H}) = I - \tilde{I}. \quad (6.36)$$

Where $\tilde{H}$ is the smoothing kernel, where in this application I use Gaussian for the smoothing step. After getting the mask using eq (6.36), the next part is summing the mask with the original image multiplied by sharpening factor a following equation :

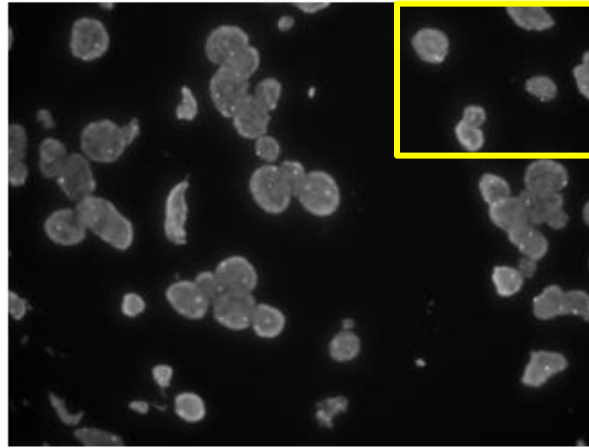
$$\check{I} \leftarrow I + a \cdot M, \quad (6.37)$$

UNSHARP MASKING RESULTS

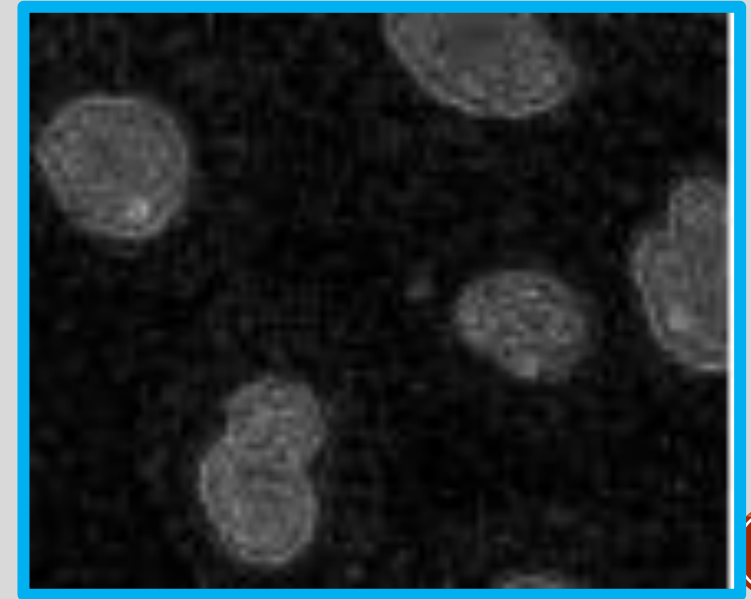
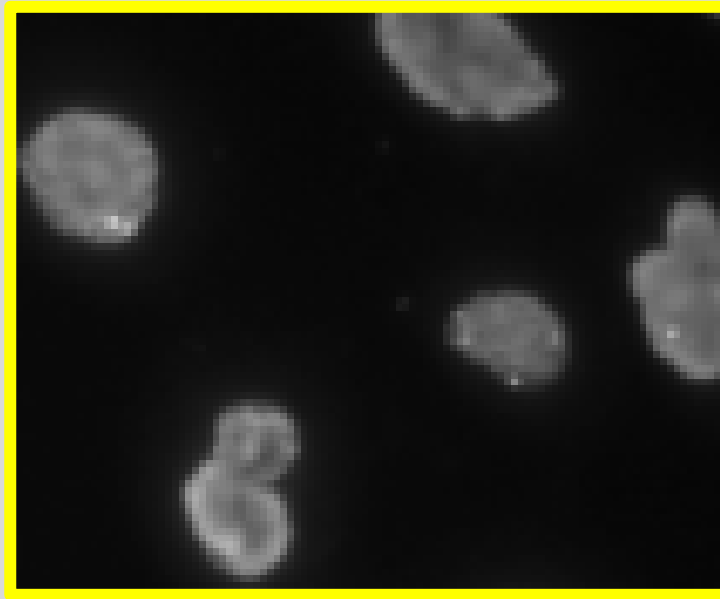
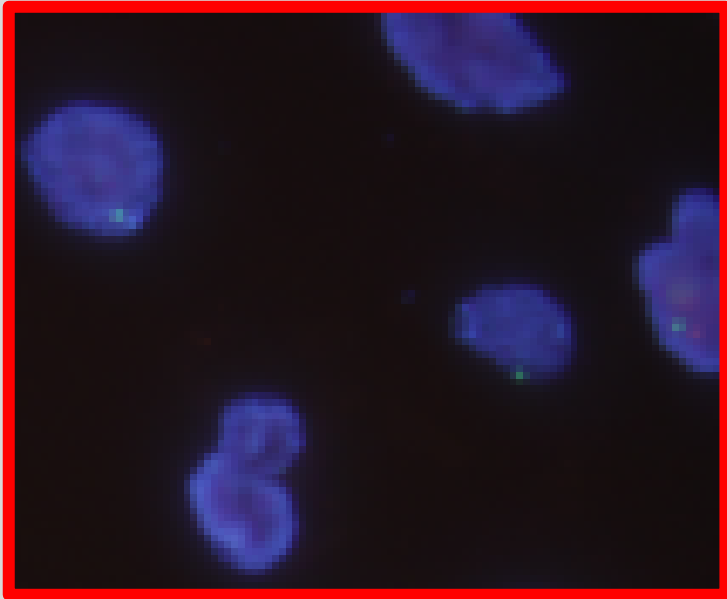
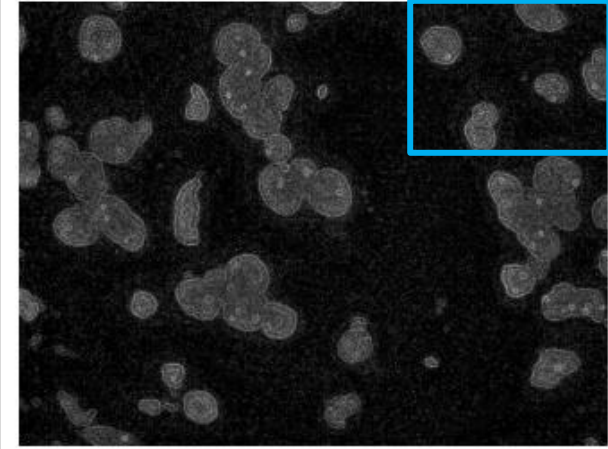
Original



Gray Image



USM



TUGAS

- Gunakan citra pada penugasan sebelumnya (week#9-simple edge operator)
- Lakukan deteksi tepi dengan menggunakan:
 - Canny edge detection (from scratch)
 - Canny edge detection (menggunakan library)
- Lakukan image edge sharpening dengan menggunakan:
 - Laplacian Operators
 - Unsharp masking
- Lakukan run-time analysis
- Bandingkan hasil dari masing-masing metode
- Siapkan presentasi yang menjelaskan tugas kalian

THANK YOU